Graph Attention Networks

Anish Bhupalam
ab2352

Jason Nair
jn582

Anagha Ram
ar2624

1 Introduction

Our project reproduces and analyzes Graph Attention Networks (GATs) for inductive node classification on graph-structured data, using the Protein-Protein Interaction (PPI) dataset. The core challenge is aggregating information from a node's neighbors, since not all neighbors contribute equally. Our goal is to evaluate whether learned attention can better prioritize informative neighbors compared to uniform aggregation.

We replicate the inductive experiment from *Graph Attention Networks* by Veličković et al. [1], which introduces an attention-based graph neural network that learns the relative importance of neighboring nodes. The model computes attention coefficients for each edge and aggregates neighbor features through a weighted sum. As a baseline, we compare against Const-GAT, which assigns uniform weights. The original work shows that combining full neighborhood aggregation with learned attention leads to strong performance on benchmarks such as PPI and citation networks.

2 Chosen Result

We aim to reproduce the paper's inductive node classification experiment on a Protein-Protein Interaction (PPI) dataset. In a PPI graph, each node represents a protein, each edge represents a biological interaction, and each node can belong to multiple classes. In the experiment, the model is trained on a set of PPI graphs and then evaluated on entirely unseen graphs at test time.

We evaluate using the micro-averaged F1 score, which balances precision and recall across all label classes. Precision measures how often the model is correct when it predicts a label, and recall measures how many of the true labels the model correctly identified. This is a more meaningful metric than plain accuracy, which would reward the model for just getting easy majority classes right.

Our target is an **F1 score of 0.973 ± 0.01 for GAT and 0.934 ± 0.01 for Const-GAT**. To make sure our results are statistically reliable, we run the model 10 times and report the average mean and standard deviation.

3 Methodology

We implemented GATs using PyTorch Geometric on Google Colab, following the original architecture while adapting to a modern framework. The core component is a graph attentional layer, where each node aggregates features from its first-order neighbors using learned, normalized attention coefficients. For each edge, we compute a raw attention score, apply a LeakyReLU activation, and normalize the scores using a softmax over the node's neighborhood to produce a weighted sum of neighbor features.

Our model uses a three-layer GAT architecture. The first two layers each consist of 4 attention heads computing 256 features, followed by ELU activations and skip connections. The final layer uses 6 attention heads computing 121 features, which are averaged and passed through a sigmoid activation for multi-label classification. As a baseline, we implement Const-GAT, which differs only in that it assigns uniform attention weights to all neighbors instead of learning them.

We conduct experiments on the Protein-Protein Interaction (PPI) dataset, which is designed for inductive node classification across multiple graphs. The model is trained using the Adam optimizer with a learning rate of 0.005, a batch size of 2 graphs, and early stopping based on validation micro-F1 with a patience of 100 epochs. To address training instability due to exploding gradients, we apply gradient clipping.

For evaluation, we use the micro-F1 score, which balances precision and recall and is standard for multi-label classification tasks like PPI. We run multiple trials for both GAT and Const-GAT to compare performance and stability. Additionally, we introduce

a modification to the original methodology by incorporating per-head residual connections, which improved performance in our implementation.

4 Results

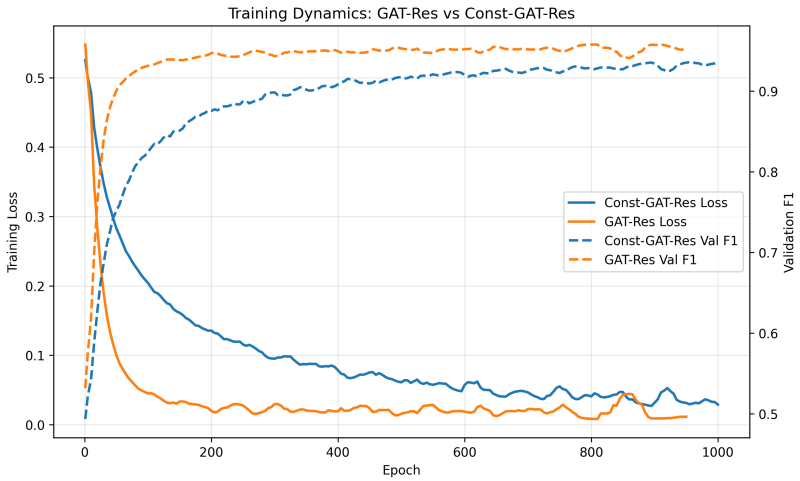

Figure 1: Training curves comparing GAT-Res and Const-GAT-Res, showing faster convergence and higher validation F1 for GAT-Res

Our implementation produced a F1 score of 0.9719 for GAT and 0.9574 for Const-GAT. Our GAT result matched the paper's result, and our Const-GAT result actually exceeded the paper. This was due to our addition of per-head residual connections, which increased F1 by 0.02. For GAT, the loss curve and F1 score converged in fewer epochs than Const-GAT due to the learned attention mechanism.

Inductive PPI Test F1 Comparison

Method	PPI Test F1
Random	0.396
MLP	0.422
GraphSAGE*	0.768
Const-GAT (PAPER)	0.934 ± 0.006
GAT (PAPER)	0.973 ± 0.002
Const-GAT-Res (OURS)	**0.9574 ± 0.0004**
GAT-Res (OURS)	**0.9719 ± 0.0024**

Figure 2: Replication of summary results of micro-averaged F1 scores for PPI dataset, based on Table 3 from Graph Attention Networks by Petar Velickovic et al.1

We compare our results to other methods presented in the paper. Random selects a random class for each node, reaching an F1 score of 0.396. A standard MLP, which only considers a node's features and no interactions, reaches an F1 score of 0.422. GraphSAGE, a previous graph neural network method that only samples the neighborhood, reached an F1 score of 0.768.

5 Conclusion

Our re-implementation of Graph Attention Networks on the PPI dataset highlights several key lessons. GAT achieves strong and consistent node classification performance with low variance across runs, confirming the effectiveness of full-neighborhood

aggregation. While GAT outperforms Const-GAT, the relatively small improvement suggests that learned attention provides incremental gains rather than fundamentally changing model behavior, indicating that attention exhibits limited adaptability and behaves closer to uniform aggregation in practice. This aligns with later work such as GATv2 [2], which identifies limitations in the original attention mechanism and proposes a more expressive formulation with dynamically varying attention. Our findings reinforce this need, highlighting a gap between the theoretical flexibility of attention and its realized impact. Future work could explore GATv2 or other dynamic attention variants, as well as evaluate these models on more diverse or noisy graphs to better understand when adaptive attention is most beneficial.

References

[1] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. *ICLR*, 2018.

[2] S. Brody, U. Alon, and E. Yahav. How attentive are graph attention networks? *ICLR*, 2022.